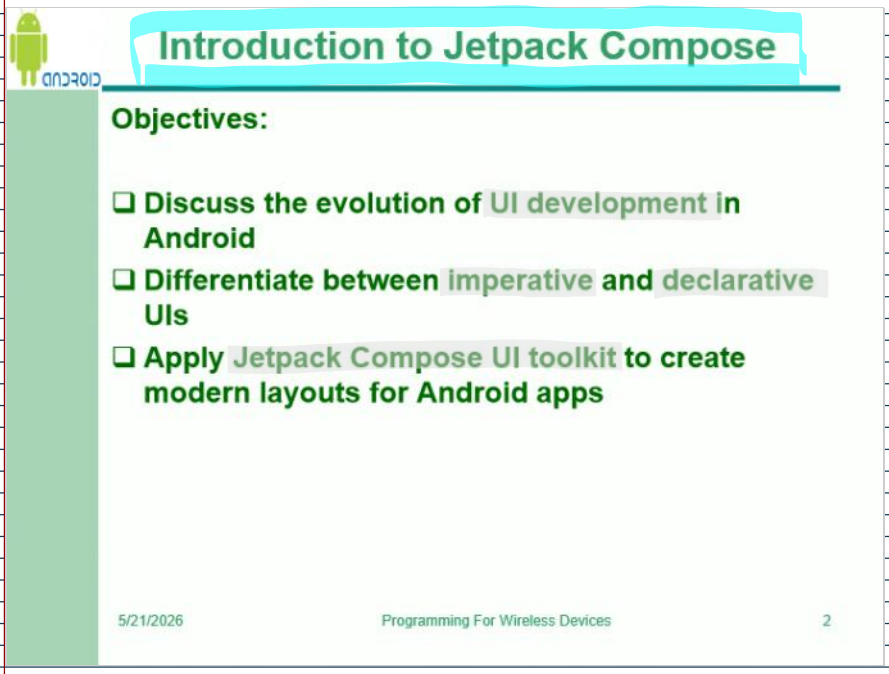
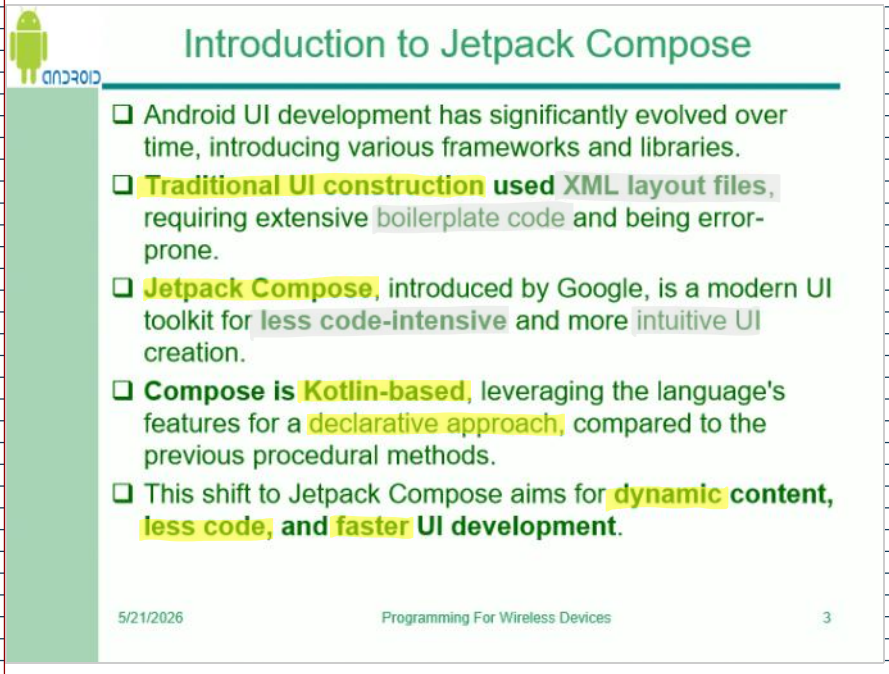
LEC 3 - Slides

Tuesday, May 26, 2026 15:50

COMP30...





- jetpack compose
- functions denoted with @composable

- traditional design is xml-based
  - requires a lot more boilerplate code, more prone to errors

Advantages of Jetpack Compose

- Jetpack Compose **reduces the need for boilerplate code**, allowing developers to achieve more with less code.
- It offers **responsive layouts** that adapt seamlessly to different screen sizes and orientations.
- **Components in Jetpack Compose are reusable**, which streamlines development across different parts of an app.

declarative
- coding with IDE help

Jetpack compose
- modern way to build AI

Imperative vs. Declarative UIs

- **Imperative UI programming involves manual handling of UI changes and state,** requiring developers to manage the lifecycle and update the UI explicitly.
  - It's step-by-step and state changes are mutable, leading to more boilerplate code.
- **Declarative UI programming**, exemplified by Jetpack Compose, **focuses on defining UI as a function of state**.
  - Developers declare what the UI should look like, and the framework handles updates automatically, leading to less code, improved readability, and easier management of UI state and lifecycle.
- **Imperative approach**: Detailed control over UI, manual state management.
- **Declarative approach**: Focus on describing UI states, not steps, immutable state in declarative UIs, framework handles UI updates automatically.

Understanding Compose's Declarative Nature

- Jetpack Compose operates on a declarative UI paradigm, where **UI elements are a function of the application state**.
- Unlike imperative UIs where you must manage state changes and UI updates, **Compose automatically rebuilds the UI when the state changes**.
- Compose's architecture and toolkit are designed to be intuitive, leveraging Kotlin's features for concise, readable code.
- By declaring UI components, developers let **Compose handle the UI's synchronization with the underlying data**, leading to a more robust and efficient way to build interfaces.

Practical Example - Counter UI in JetPack Compose

```
@Composable
fun MyApp() {
    var count by remember { mutableStateOf(0) }
    Column(modifier = Modifier.padding(16.dp)) {
        Text(text = "Counter: $count", style =
MaterialTheme.typography.bodyLarge)
        Spacer(modifier = Modifier.height(16.dp))
        Button(onClick = { count++ }) {
            Text("Increment")
        }
    }
}
```

- Explanation of state management and composables.

5/21/2026 Programming For Wireless Devices 7

(tested this in class)

XML vs. JetPack Compose for a Counter UI

- xml code:

```
<TextView android:id="@+id/counterTextView" android:text="Counter: 0"
/>
<Button android:id="@+id/incrementButton" android:text="Increment" />
```

- Kotlin code:

```
val counterTextView: TextView = findViewById(R.id.counterTextView)
val incrementButton: Button = findViewById(R.id.incrementButton)
incrementButton.setOnClickListener {
    count++
    counterTextView.text = "Counter: $count"
}
```

- Comparison of verbosity and complexity between XML and Compose

5/21/2026 Programming For Wireless Devices 8

Composable Functions - The Building Blocks

- **Composable functions** are the fundamental building blocks of UI in Jetpack Compose, **marked by the @Composable annotation**.
  - This annotation **informs the Compose compiler that the function is meant to define part of the UI**.
  - **It enables the function to use other composables** internally and maintain a reactive relationship with app data.
- **When state changes**, any **composable functions that rely on that state are automatically reinvoked**, ensuring the UI reflects current data without manual intervention.
  - This streamlines the process of creating dynamic and interactive UIs.

5/21/2026 Programming For Wireless Devices 9

Code Example: Simple Composable Function

```
import androidx.compose.material.Text
import androidx.compose.runtime.Composable
import androidx.compose.ui.tooling.preview.Preview

@Composable
fun Greeting(name: String) {
    Text(text = "Hello, $name!")
}

@Preview
@Composable
fun PreviewGreeting() {
    Greeting(name = "Jetpack Compose")
}
```


Previewing Composables

- The **@Preview annotation** in Jetpack Compose is a powerful tool that **helps developers visualize their UI components** without the need to deploy the app to a device or emulator.
  - When applied to a composable function, it **allows Android Studio to generate a live preview of the UI**, showing how it will appear based on the current code.
  - This **facilitates rapid iteration and design adjustments**, enabling developers to experiment with UI changes quickly and see the effects immediately in the IDE.

```
@Preview(showBackground = true)
@Composable
fun PacktPublishingPreview() {
    PacktPublishing("Android Development with Kotlin")
}
```


* takes long to initialize *
- allows to view output of composable w/o running the emulator

Modifiers in Compose

- In Jetpack Compose, modifiers are **used to configure and customize the layout and appearance of UI components**.
  - Modifiers allow you to **apply common layout parameters** such as padding, margins, alignment, and sizing.
  - They also **handle more complex functionalities like click handling**, scrolling behaviors, and animations.
- By **chaining multiple modifiers**, you can define a composable's appearance and behavior in a flexible and declarative way.
  - This approach **streamlines the UI code and enhances readability and maintainability** by keeping layout configurations concise and localized to each component.

modifier objects
- allow customization of layout and appearance of UI
- ex: layout parameters
  - padding
  - margins
  - alignment
  - sizing
- can chain multiple for animations, click handling, scrolling, etc

Code Example: Applying Modifiers

```
import androidx.compose.material.Text
import androidx.compose.runtime.Composable
import androidx.compose.ui.Modifier
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.text.TextStyle
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
@Composable
fun StyledText() {
    Text(
        text = "Hello, Compose!",
        //adds padding around the text
        modifier = Modifier.padding(24.dp),
        style = TextStyle(
            color = Color.Blue,
            fontSize = 18.sp,
            fontWeight = FontWeight.Bold
        )
    )
}
```

== pixels

== pt (like fonts)

13

Composable Layouts Overview

- In Jetpack Compose, several layout types help structure your UI components effectively:
  - **Column**: Arranges items vertically in a single column.
  - **Row**: Places items horizontally in a single row.
  - **Box**: Layers its children on top of each other, allowing for overlapping components.
  - **ConstraintLayout**: Provides flexible positioning of child composables relative to each other.
  - **LazyColumn** and **LazyRow**: For efficient scrolling lists, loading items lazily as they become visible.
  - Grids (**LazyVerticalGrid** and **LazyHorizontalGrid**): Display items in a grid layout, either vertically or horizontally.
- **These layouts can be enhanced** with various modifiers for sizing, spacing, and alignment, providing a versatile framework for building complex UIs.

14

Column Layout Explained

- In Jetpack Compose, the Column layout is used to **arrange child composables vertically**.
  - It **stacks components** such as text, buttons, images, or other custom composables **in a top-down fashion**.
  - This layout is versatile for creating forms, lists, or any vertical grouping of elements within your app.
- You **can customize the spacing, alignment, and arrangement of children** within a Column using modifiers and parameters like verticalArrangement and horizontalAlignment to control the exact placement and distribution of the children.

Code Example: Column Layout

```
import androidx.compose.foundation.layout.Column
import androidx.compose.material.Text
import androidx.compose.runtime.Composable
import androidx.compose.ui.tooling.preview.Preview

@Composable
fun GreetingColumn() {
    // stack Text elements vertically
    Column {
        Text(text = "Hello, Jetpack Compose!")
        Text(text = "This is the second line.")
        Text(text = "And here's the third.")
    }
}
// see the layout directly in Android Studio before running it on a device or emulator
@Preview
@Composable
fun PreviewGreetingColumn() {
    GreetingColumn()
}
```

1 column in 3 rows

Row Layout Explained

- The Row layout in Jetpack Compose is **used for arranging child components horizontally**.
- This layout is **similar to a horizontal LinearLayout** in traditional Android development, allowing developers to **place elements such as text, buttons, images, and other composables side by side across the horizontal axis**.
  - It's particularly **useful for creating toolbars, horizontal menus**, or any user interface that requires inline placement of elements.
- You can **control the alignment, spacing, and distribution of these elements using various modifiers** and parameters tailored to fit the design needs.

Code Example: Row Layout

```
import androidx.compose.foundation.layout.Row
import androidx.compose.material.Text
import androidx.compose.material.Button
import androidx.compose.runtime.Composable
import androidx.compose.ui.tooling.preview.Preview

@Composable
fun ButtonRow() {
    Row { // arrange the buttons horizontally
        Button(onClick = { /* Handle click */ }) {
            Text("Button 1")
        }
        Button(onClick = { /* Handle click */ }) {
            Text("Button 2")
        }
    }
}
//render this layout in Android Studio's design view
@Preview
@Composable
fun PreviewButtonRow() {
    ButtonRow()
}
```

Box Layout for Overlap and Alignment

- The Box layout in Jetpack Compose is **used for layering components on top of each other**, allowing for overlapping UI elements within a single container.
- This layout is particularly **useful for creating complex UIs** where elements like badges, buttons, or custom graphics need to overlay other components.
- You can **control the alignment of child components within the Box using alignment parameters**, enabling precise positioning of each element, whether centered, aligned to a corner, or stretched across the container.
- This makes Box ideal for **designing intricate layouts with elements that need specific placement relative to one another**.

Code Example: Box Layout

```
// Add necessary import statements

@Composable
fun IconTextOverlay() {
    // create a container with a light gray background
    Box(
        modifier = Modifier.size(150.dp).background(Color.LightGray).padding(10.dp),
        contentAlignment = Alignment.Center
    ) {
        Text("Look here!", Modifier.align(Alignment.Center))
        // Icon component aligned to the top end of the Box, overlapping the text
        Icon(
            imageVector = Icons.Filled.Favorite,
            contentDescription = "Favorite",
            modifier = Modifier.align(Alignment.TopEnd).size(40.dp)
        )
    }
}

@Preview
@Composable
fun PreviewIconTextOverlay() {
    IconTextOverlay()
}
```

box contents

Box( ) { }

box formatting

content inside the Box

Lists in Compose - LazyColumn & LazyRow

- In Jetpack Compose, LazyColumn and LazyRow are components **used for displaying scrollable lists** efficiently.
  - **LazyColumn arranges items vertically**, similar to a RecyclerView with a LinearLayoutManager
  - **LazyRow** arranges items horizontally.
- These components are **"lazy" because they only render the visible items on the screen**, optimizing performance and resource usage.
- This makes them **ideal for handling large datasets or lists** where only a subset of items is displayed at a time.

Code Example: LazyColumn

```
@Composable
fun SimpleLazyColumn() {
    val itemList = List(100) { "Item #$it" } // Generates a list of 100 items

    LazyColumn {
        items(itemList) { item ->
            Text(text = item)
        }
    }
}

@Preview
@Composable
fun PreviewSimpleLazyColumn() {
    SimpleLazyColumn()
}
```


: Grids in Compose - LazyVerticalGrid & LazyHorizontalGrid

- In Jetpack Compose, **LazyVerticalGrid** and **LazyHorizontalGrid** enable **efficient grid layouts for items, adjusting vertically or horizontally**.
  - **LazyVerticalGrid** structures items in multi-column grids, perfect for displaying collections like photos, **scrolling vertically**.
  - **LazyHorizontalGrid** sets items in **horizontal rows**, ideal for galleries or carousels.
- Both grids **optimize performance by rendering only visible elements**, making them suitable for large datasets.

LazyVerticalGrid – Code Example

```
LazyVerticalGrid( modifier = Modifier
.fillMaxSize()
.background(Color.LightGray)
.padding(8.dp),
columns = GridCells.Fixed(3)
) {
items(100) { Text(
modifier = Modifier
.padding(8.dp),
text = "Item number $it"
)
}
}
```

LazyHorizontalGrid – Code Example

```
LazyHorizontalGrid( modifier = Modifier
.fillMaxSize()
.background(Color.LightGray)
.padding(8.dp),
rows = GridCells.Fixed(3)
) {
items(100) { Text(
modifier = Modifier
.padding(8.dp),
text = "Item number $it"
)
}
}
```


ConstraintLayout for Complex UIs

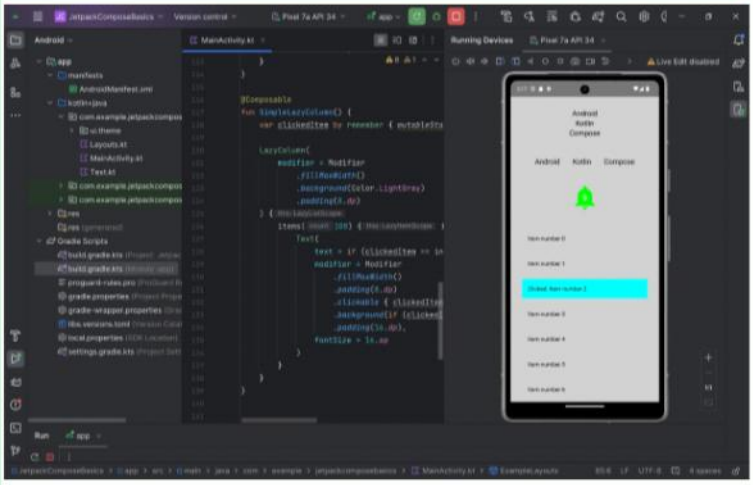
- In Jetpack Compose, ConstraintLayout offers a powerful way to build complex and responsive UIs by allowing developers to **create flexible layouts with relative positioning**.
  - It provides **advanced layout capabilities** where you can define constraints for layout elements relative to each other or to the parent container.
  - This **facilitates the creation of scalable UIs that adjust smoothly across different screen sizes and orientations**, making it ideal for intricate designs that require precise alignments and adjustments.

ConstraintLayout – Code Example

```
ConstraintLayout( modifier = Modifier
.padding(16.dp)
) {
val (icon, text) = createRefs() Icon(
modifier = Modifier
.size(80.dp)
.constrainAs(icon) { top.linkTo(parent.top) bottom.linkTo(parent.bottom)
start.linkTo(parent.start)
},
)
Text(
imageVector = Icons.Outlined.Notifications, contentDescription = null,
tint = Color.Green

modifier = Modifier
.constrainAs(text) { top.linkTo(parent.top) bottom.linkTo(parent.bottom) start.linkTo(icon.end) },
text = "9",
style = MaterialTheme.typography.titleLarge
)
}
```

JetpackComposeBasics Example



References

- Textbook Chapter 3
- https://developer.android.com/develop/ui/compose
- https://developer.android.com/develop/ui/compose/tutorial
- https://developer.android.com/develop/ui/compose/kotlin
- Android Documentation